Cours : M1INRA1-Programmation Web

Paul-Antoine Bisgambiglia

Déroulé du cours de Programmation Web (Intro DFS) :

L'objectif du cours du semestre est de vous faire comprendre les différentes méthodes de développement web et de renforcer vos compétences dans le domaine.
Ce document définit les exigences de qualité et la trajectoire pédagogique. Il vous guide pour passer d'une approche "code-first" vers une méthodologie rigoureuse "engineering-first", centrée sur la sécurité, l'accessibilité et la performance.

Vous aurez une note de participation, c'est-à-dire de présentation de vos codes, une note de TP**, le sujet est en fin** de mail et une note d'examen sur papier. L'examen aura lieu en janvier, c'est du commentaire de code et/ou des questions sur des définitions de cours.

Table des matières

I. Déroulement des cours : 3

II. Les types de programmation web : 8

III. Le roulement : 8

IV. TD : "Crawl & Rank" – Construire un Moteur d'Indexation Minimal 10

A. 1. Introduction et Objectifs du Module 10

B. 2. Étape 1 : Architecture et Conception du Robot (Le Crawler) 10

1. Concept : Web Spiders & Crawlers 10

C. 3. Étape 2 : Extraction et Analyse Sémantique (HTML5 & SEO) 11

1. Concept : Sémantique HTML5 11

D. 4. Étape 3 : Développement de l'Indexeur et Persistance (Le Backend) 11

1. Concept : Le Modèle MVC 12

E. 5. Étape 4 : Interface de Recherche et Expérience Utilisateur (UI/UX) 12

1. Concept : Accessibilité (a11y) 12

F. 6. Étape 5 : Validation, Déploiement et Audit SEO 12

V. Sujet de TD : Maîtriser les Fondamentaux de Laravel 14

A. 1. Introduction à l'Architecture Web et au Framework Laravel 14

1. Analyse du cycle de vie d'une requête 14

2. Concept de Base : Le pattern MVC (Model-View-Controller) 14

B. 2. Étape 1 : Initialisation et Configuration de l'Environnement 14

1. Installation pratique 15

2. Gestion de la configuration et sécurité 15
3. Concept de Base : Les Dépendances 15
C. 3. Étape 2 : Conception de la Base de Données et Modélisation 15
1. Mise en pratique : Création d'un module de tâches 15
2. L'ORM Eloquent et la sécurité 16
D. 4. Étape 3 : Logique Applicative (Routage et Contrôleurs) 16
1. Définition des routes et logique 16
2. Concept de Base : Verbes HTTP et Sémantique 16
E. 5. Étape 4 : Interface Front-End et Interactions 17
1. Structure sémantique avec Blade 17
2. Ergonomie et Sécurité des formulaires 17
3. Concept de Base : Responsive Web Design 18
F. 6. Conclusion et Checklist de Validation 18
1. Checklist de Sécurisation et de Qualité 18
VI. TD API platform : Construction d'une Application Full-Stack avec API Platform 19
A. 1. Introduction et Fondamentaux de l'Architecture Web 19
1. Analyse Stratégique : Monolithe vs Architecture Découplée 19
2. Concepts de Base 19
B. 2. Initialisation de l'Environnement et Installation 19
1. Guide Technique d'Installation 19
2. Concepts de Base 20
C. 3. Modélisation des Données et Persistance (ORM) 20
1. Mise en Pratique : Création d'Entités 20
2. Analyse Stratégique : Performance et Persistance 20
3. Concepts de Base 20
D. 4. Logique Applicative : Opérations et Contrôleurs 21
1. Développement et Routage Déclaratif 21
2. Validation : Le miroir de sécurité 21
3. Concepts de Base 21
E. 5. Interaction avec le Front-End et Expérience Utilisateur (UX) 21
1. Accessibilité et Structure Sémantique 21
2. Analyse Stratégique : Les Critères de Bastien & Scapin 22
3. Pro-Tips Pédagogiques 22
F. 6. Sécurisation, SEO et Finalisation 22
1. Checklist de Mise en Production (Standards Industriels) 22
2. Analyse "So What?" : Performance et Business 22

3. Concepts de Base 22
VII. Sujet du TP noté : 24

I. Déroulement des cours :

Le projet fil rouge est le développement étape par étape d'une application de site e-Commerce, il faudra faire une interface pour les clients qui souhaitent acheter un produit et une interface administrateur pour que le personnel de l'entreprise puisse mettre en vente ses produits.

Séance du 28/9

Introduction du module // cours d'intro à l'UX/UI // exercices.
Exercice fil rouge => coder un système de login avec soit le backend en PHP sans framework, soit avec Laravel ou API plateform (Laravel et API plateform sont expliqués à la fin du document en sujet de TD). L'exercice est à faire seul. Vos codes doivent être fonctionnels, testés, commentés et déposés sur un GITxxx. **Ils seront présentés sous forme de revue de code le 01/10 (cours suivant).**

Info base de données à utiliser :

```
CREATE TABLE `2026_M1`.`user` (`user_id` INT NOT NULL AUTO_INCREMENT , `user_login` TEXT NOT NULL , `user_password` LONGTEXT NOT NULL , `user_compte_id` INT NOT NULL , `user_mail` TEXT NOT NULL , `user_date_new` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP , `user_date_login` TIMESTAMP on update CURRENT_TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP , PRIMARY KEY (`user_id`), UNIQUE `cle-etrangere` (`user_compte_id`)) ENGINE = InnoDB;
```

Les dates sont gérées automatiquement, le champ 'user_compte_id' va permettre de faire le lien avec le compte client (clé étrangère) de l'utilisateur. Le compte client sera à ajouter par la suite pour gérer le panier et les commandes.

Conception & Expérience Utilisateur (UX/UI)

- **Livrables :** Personas, Story-boards, Wireframes (Figma/Proto.io).
- **Standards Ergonomiques (Bastien & Scapin) :**
  - *1.1 Incitation :* Guider la saisie (ex: format de date `jj/mm/aa`).
  - *1.3 Feedback immédiat :* Informer l'utilisateur de l'état des traitements (loaders, toasts).
  - *2.1 Brièveté :* Appliquer la **"Règle des 3 clics"** pour l'accès à l'information et limiter chaque ligne de texte à **90 caractères** maximum pour garantir la lisibilité.

- **Accessibilité (a11y) :** Respect des 4 piliers (Perceptible, Utilisable, Compréhensible, Robuste). Utilisation impérative de Lighthouse et axe DevTools pour valider la conformité WAI-ARIA et les contrastes.

Fondations Front-End (HTML5/CSS3 & JS ES6+)

- **Conventions HTML5 :** Déclaration `<!DOCTYPE html>`, `charset="utf-8"`, et usage exclusif des balises sémantiques (`<main>`, `<nav>`, `<article>`, `<section>`).
- **Mise en page : Proscrire les flottants**. Mandater Flexbox et Grid Layout pour un design responsive fluide.
- **Validation & Interactivité :** Validation client via attributs `required` et `pattern`.
    - *Exemple de rigueur :* Utilisation de Regex spécifiques pour les codes pays (`[A-Za-z]{3}`) et les politiques de mots de passe complexes (`(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}`).

Séance du 01/10

Présentation des codes // Cours HTML - CSS // Exercices.
Exercice fil rouge => coder un CRUD pour une gestion des utilisateurs et des produits (ajouter un produit, le supprimer, le modifier).
Exercice à faire en groupe de 6 en suivant les modes de développement :
1. Mode API, (2 membres du groupe)
2. Mode MVC, (2 membres du groupe)
3. Mode classique, (2 membres du groupe)
Les codes seront présentés le 13/10.

Backend PHP & Modélisation de Données

- **Architecture :** Schéma SQL normalisé (Utilisateurs/Produits).
- **Sécurisation :** Centralisation des accès DB via une instance PDO unique. Configuration impérative du `viewport` et des meta-données de base (`description`, `author`, `copyright`).

Fonctionnalités Métier & Qualité Logicielle

- **Gestion de contenu :** Implémentation d'un BackOffice sécurisé permettant le CRUD complet du catalogue.
- **Fiabilité des données :** Utilisation de Faker pour les tests de charge et de filtrage.
- **Logique Panier :** Calcul dynamique (TVA, frais d'expédition) avec validation côté serveur pour éviter la manipulation des prix côté client.

```
CREATE TABLE produit (
   id_p INT AUTO_INCREMENT PRIMARY KEY,
   type_p VARCHAR(100) NOT NULL,
   designation_p VARCHAR(255) NOT NULL,
   prix_ht DECIMAL(10,2) NOT NULL,
   date_in DATE NOT NULL,
   times_in TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ,
   stock_p INT DEFAULT 0
);
```

Vous pouvez compléter cette table, pour gérer l'historique des prix d'un produit et aussi proposer une promotion, il faut donc ajouter un champ pourcentage de promotion (ppromo), s'il n'est pas complété le produit est à afficher à son prix hors taxe et s'il y a une valeur, on affiche la promotion et on calcule le nouveau prix.
Ajouter aussi la possibilité de déposer une image.
Si aucune image n'est liée au produit, afficher une image générique.

Un produit est un élément à vendre sur votre site, vous pouvez choisir les désignations. Vous devez ajouter des fonctions pour gérer les produits.

Pour les types de produit, vous devez faire un fichier .json coté serveur avec la liste des types (categorie) et côté client, il faudra afficher un select html.

```
[
   {
      "id": 1,
      "categorie": "Smartphone"
   },
   {
      "id": 2,
      "categorie": "Ordinateur portable"

   },
   {
```

```
    "id": 3,
    "categorie": "Casque audio"
  },
  {
    "id": 4,
    "categorie": "Montre connectée"
  },
  {
    "id": 5,
    "categorie": "Tablette"
  },
  {
    "id": 6,
    "categorie": "Caméra"
  }
]
```


Pour le développement en API, voici les fonctions à implémenter :

GET : La méthode GET demande une représentation de la ressource spécifiée. Les requêtes GET doivent uniquement être utilisées afin de récupérer des données.
HEAD : La méthode HEAD demande une réponse identique à une requête GET pour laquelle on aura omis le corps de la réponse (on a uniquement l'en-tête).
POST : La méthode POST est utilisée pour envoyer une entité vers la ressource indiquée. Cela entraîne généralement un changement d'état ou des effets de bord sur le serveur.
PUT : La méthode PUT remplace toutes les représentations actuelles de la ressource visée par le contenu de la requête.
DELETE : La méthode DELETE supprime la ressource indiquée.

Séance du 13/10

Point sur l'exercice fil rouge et point cours

Séance du 19/10

Présentation des codes // Cours SEO // exercices.

Exercice de cours => **faire un crawler (robot d'indexation) avec CURL ou bs4 (https://www.scrapingbee.com/blog/crawling-python/)**

Exercice fil rouge => présentation des codes de CRUD et **premier roulement :**

- **pour le cours suivant => Ajouter dans le CRUD les fonctionnalités suivantes : système de Sign Up / Log In (couplage des codes du premier cours avec le code de votre CRUD).**

Les codes seront présentés le 10/11.

Déploiement du Système de Roulement & Contrats

- **Transition technique :** Lors du passage au mode API REST, les binômes doivent impérativement livrer un **JSON Schema** (Contrat d'interface).
- **Documentation :** Chaque rotation nécessite une "Code Review" formalisée. Le code doit être documenté (PHPDoc/JSDoc) pour assurer la transmission de la connaissance sans perte d'information technique.

SEO, Performance & Mise en Production

- **Audit SEO & Technique :**
  - Hiérarchie `h1-h6` stricte (un seul `h1` par page).
  - Optimisation des images : Format WebP privilégié, attribut `alt` obligatoire, et définition systématique des dimensions (`width/height`) en CSS pour que le navigateur réserve l'espace (évite le CLS).
  - Fichiers serveurs : `robots.txt` (avec `Disallow: /admin`) et `sitemap.xml`.
- **Métadonnées étendues :** Inclusion des tags `Distribution`, `Rating` et `Language`.
- **Performance :** Analyse obligatoire des **Core Web Vitals** (LCP pour l'affichage, FID pour la réactivité, CLS pour la stabilité visuelle).

Séance du 22/10 :

point sur l'exercice fil rouge et point cours

Exercice fil rouge => présentation des codes de CRUD et ajout d'un **BackOffice** (Une interface pour gérer les utilisateurs et les produits).

Séance du 9/11 :

Cours API // exercices.
Exercice fil rouge => présentation des codes « Sign Up / Log In »

- **pour le cours suivant => Ajouter les fonctionnalités suivantes : système de Trie par type de produit (ajouter des objets avec faker https://www.docstring.fr/blog/generer-des-donnees-aleatoires-avec-faker/)**
- **Deuxième roulement.**
- Faire des tests avec selenium https://www.selenium.dev/

Séance du 12/11 :

TD Laravel + dans votre code ajouter une API pour créer un tunnel de vente fictif.

Séances du 19/11 et 7/12 :

Point sur l'exercice fil rouge et point cours
Exercice fil rouge => ajouter un panier, il doit être possible d'ajouter des produits, de modifier la quantité et de voir l'évolution dynamique du montant, en ajoutant la TVA et des frais d'expédition.

Séance du 9/12 :

Cours JS + exercice
Exercice fil rouge => présentation des codes de « Trie »

Séances du 15/12 et 17/12 :

Présentation et **notes** des codes de TP (le sujet de TP est en fin de mail)

II. Les types de programmation web :

Il y a trois types de programmation : 1. Mode API, 2. Mode MVC, 3. Mode classique.
Le Mode API implique de développer un backend qui suit les recommandations REST ou SOA. Il s'agit de développer plusieurs fonctions qui seront appelées par une méthode http (POST, GET, UPDATE, PUT, etc.). Elles devront renvoyer un fichier XML, JSON ou du texte.
Le Mode MVC implique de suivre ce pattern de conception avec un modèle qui gère les interactions avec la base de données, un contrôleur qui fait les liens entre les vues et les modèles et des vues consultables par l'utilisateur.
Le mode classique implique de coder sans modèle, les interactions entre requêtes du client et la base de données peuvent être codées dans les mêmes fichiers.

III. Le roulement :

Vous allez vous regrouper par 6, en trois équipes de 2, et dans chaque équipe faire 3 binômes.

Explication par l'exemple, pour chaque équipe :

- Le binôme 1 va faire le premier exercice en développant une API (1. Mode API)
- Le binôme 2 va faire le premier exercice en suivant le pattern MVC (2. Mode MVC)
- Le binôme 3 va faire le premier exercice en développant sans modèle from scratch (3. Mode classique)

A chaque nouveau cours, vous allez vous transmettre vos codes afin de travailler sur toutes les approches de programmation web, c'est ça le roulement, et c'est pour ça que le code est présenté en cours et qu'il doit être bien commenté.

IV. TD : "Crawl & Rank" – Construire un Moteur d'Indexation Minimal

A. 1. Introduction et Objectifs du Module

Dans l'écosystème numérique actuel, la visibilité d'une plateforme ne dépend pas seulement de son esthétique, mais de sa capacité à être décodée par les algorithmes de recherche. Statistiquement, 93 % des expériences en ligne débutent par un moteur de recherche, ce qui place la compréhension des robots d'indexation au fondement même du développement web moderne. Un développeur senior ne conçoit plus de simples interfaces ; il bâtit des architectures capables d'exposer l'information de manière structurée à des agents automatisés. Ce module propose de construire un indexeur minimal pour illustrer que la performance SEO est une problématique de structure logicielle avant d'être une affaire de marketing.

- **Objectifs pédagogiques :**
  - Concevoir une architecture client-serveur robuste et scalable.
  - Intégrer la dimension SEO et sémantique dès la première ligne de code.
  - Appliquer des standards de sécurité industriels (OWASP Top 10).

Le fonctionnement d'un robot repose sur la simulation rigoureuse du cycle de vie d'une requête web. Ce processus séquentiel débute par la saisie d'une URL, suivie d'une résolution DNS et d'une connexion TCP/TLS. Le robot émet ensuite une requête HTTP (méthode `GET`), le serveur traite la demande en interrogeant souvent une base de données via SQL, puis renvoie une réponse (HTML/JSON) que le robot déconstruit. Avant de coder cet outil, il est impératif de comprendre que le terrain de jeu de l'indexeur est défini par l'architecture même du Web.

B. 2. Étape 1 : Architecture et Conception du Robot (Le Crawler)

Le choix de l'architecture est la décision initiale la plus critique pour assurer la scalabilité. Pour un indexeur, on oscillera entre un monolithe efficace pour les petits volumes ou une architecture en microservices pour distribuer la charge de crawling. Le robot doit être conçu pour parcourir le web sans saturer les ressources du système, tout en respectant scrupuleusement la hiérarchie de consultation des fichiers de configuration du serveur.

Analyse du fonctionnement Le processus de crawling est une boucle récursive rigoureuse :

1. Analyse prioritaire du fichier `.htaccess`, puis téléchargement du fichier `robots.txt`.
2. Chargement de la racine du site (URL cible).
3. Extraction systématique des liens internes via les balises `<a>`.
4. Exécution récursive du téléchargement pour chaque nouvelle URL découverte.

1. Concept : Web Spiders & Crawlers

Les robots suivent le protocole **RES (Robots Exclusion Standard)**. Ce standard s'appuie sur le fichier `robots.txt` situé à la racine. Via des directives comme `Allow` ou `Disallow`, il définit quels répertoires sont accessibles et permet de filtrer les comportements selon les catégories de bots (User-agent).

Instructions de sécurisation D'un point de vue éthique et technique, l'implémentation d'un *Rate Limiting* (limitation du débit) est non-négociable. Sans ce middleware de contrôle, votre

robot pourrait saturer le serveur cible et déclencher involontairement une attaque par déni de service (DoS). Une fois que le contenu brut est capturé, le robot doit dépasser la simple accumulation de données pour "lire" intelligemment les informations.

C. 3. Étape 2 : Extraction et Analyse Sémantique (HTML5 & SEO)

Il existe une corrélation absolue entre la structure HTML5 et la qualité de l'indexation. Une sémantique défaillante rend un site invisible, quel que soit le talent déployé sur le design. Le code doit servir de guide au robot en hiérarchisant l'information selon son importance stratégique.

Analyse des balises critiques

Balise	Importance SEO	Rôle pour le Robot
`<title>`	Critique	Décrit le sujet central ; maximum 100 caractères.
`<meta name="robots">`	Critique	Attributs essentiels : `index`, `noindex`, `follow`, `nofollow`.
`<meta description>`	Haute	Résumé affiché dans les résultats de recherche (SERP).
`<h1>` à `<h6>`	Haute	Structure et hiérarchise le contenu textuel.
`alt` (Images)	Moyenne	Permet l'indexation du contenu visuel et l'accessibilité.

1. Concept : Sémantique HTML5

Pour garantir une structure logique, l'utilisation des balises de structure est impérative :

- `<header>` : Zone d'en-tête (site ou section).
- `<nav>` : Blocs de navigation principaux.
- `<article>` : Contenu autonome indexable.
- `<section>` : Regroupement thématique.
- `<footer>` : Pied de page (informations de copyright/contact).

Conseils de bonnes pratiques Il est impératif de limiter l'usage à une seule balise `<h1>` par page afin de clarifier le sujet principal pour l'indexeur. L'usage de balises comme `<strong>` permet de souligner les mots-clés prioritaires. Ces données, une fois extraites et hiérarchisées, doivent être stockées de manière structurée pour être exploitables par le moteur de recherche.

D. 4. Étape 3 : Développement de l'Indexeur et Persistance (Le Backend)

Le backend agit comme le cerveau du système, transformant les flux de données brutes en un index interrogeable. Le choix des technologies dépend de l'envergure du projet : Node.js avec Express pour la réactivité, ou des frameworks robustes comme Python (Django, FastAPI) ou Java (Spring Boot) pour des applications d'entreprise.

Logique d'indexation L'indexeur doit isoler les mots-clés stratégiques (ceux tapés en volume suffisant par les utilisateurs) et les lier aux URLs correspondantes. On privilégiera **PostgreSQL** pour sa gestion rigoureuse des relations de données ou **MongoDB** pour la flexibilité du stockage de documents non structurés.

1. Concept : Le Modèle MVC

Le modèle **MVC (Model-View-Controller)** est un standard architectural de presque 50 ans qui sépare les responsabilités :

- **Model (M)** : Gère les données métier et la persistance.
- **View (V)** : Responsable de l'affichage du contenu.
- **Controller (C)** : Traducteur traitant les entrées utilisateur pour mettre à jour le modèle ou la vue.

Sécurisation du Backend En tant que Lead Developer, vous devez exiger une **sanitisation** stricte des entrées et l'utilisation de **requêtes paramétrées** pour rendre les injections SQL impossibles. Pour la gestion des utilisateurs, le **hashing avec salage** est le standard de sécurité requis. Enfin, l'utilisation de l'encodage **UTF-8** (`<meta charset="utf-8">`) est obligatoire pour supporter l'intégralité des caractères internationaux sans corruption. Une fois ces données sécurisées, l'étape suivante consiste à concevoir l'interface de consultation.

E. 5. Étape 4 : Interface de Recherche et Expérience Utilisateur (UI/UX)

L'efficacité d'une interface de recherche repose sur sa sobriété. L'architecture de contenu doit respecter la règle des **4 clics maximum** : toute information doit être accessible depuis la page d'accueil en moins de quatre interactions. De plus, la performance est désormais un critère de classement direct via les **Core Web Vitals** de Google :

- **LCP (Largest Contentful Paint)** : Mesure de la vitesse d'affichage du contenu principal.
- **FID (First Input Delay)** : Mesure de la réactivité lors de la première interaction.
- **CLS (Cumulative Layout Shift)** : Mesure de la stabilité visuelle de la page.

Construction du formulaire Un formulaire performant utilise la balise `<form>` avec la méthode `GET`, rendant les termes de recherche visibles dans l'URL pour faciliter le partage. L'usage de l'attribut `placeholder` et de la mention `required` guide l'utilisateur, tandis que les lignes de texte ne doivent jamais dépasser **90 caractères** pour prévenir la fatigue visuelle.

1. Concept : Accessibilité (a11y)

Rendre le web utilisable par tous est une obligation légale et technique. Elle repose sur :

- Des contrastes de couleurs élevés (vérifiables via Lighthouse).
- L'utilisation systématique de labels de formulaires explicites (`<label for="...">`).
- Une structure compatible avec les technologies d'assistance.

Différenciateurs Clés Le *Responsive Web Design* via les Media Queries est crucial, Google pratiquant l'indexation mobile-first. Lors de la phase de conception, l'utilisation d'outils de prototypage comme **Figma**, **JustinMind** ou **Proto.io** permet de valider ces flux avant le développement. Le projet doit enfin être validé par un audit final avant sa mise en production.

F. 6. Étape 5 : Validation, Déploiement et Audit SEO

Avant tout déploiement automatisé via une chaîne **CI/CD**, une phase de test garantit la stabilité du système. Un code non validé est une dette technique immédiate.

Audit final Utilisez les standards industriels pour valider votre conformité :

- **W3C Validator** : Vérification de la syntaxe HTML/CSS.
- **Google Search Console** : Test de la crawlabilité du `robots.txt` et du sitemap.
- **Lighthouse** : Audit de performance, d'accessibilité et de SEO.

Checklist de Sécurité Finale Les éléments suivants sont non-négociables pour un déploiement professionnel :

- **HTTPS** : Chiffrement obligatoire de tous les flux.
- **Headers CORS** : Contrôle strict des accès cross-origin.
- **Protection XSS & CSRF** : Sécurisation des cookies, des sessions et sanitisation des sorties.
- **Validation par patterns** : Utilisation de l'attribut `pattern` (Regex) pour les champs sensibles.

Le mot de la fin Ce projet de moteur d'indexation minimal prouve que le SEO n'est pas une surcouche optionnelle, mais une architecture de code réfléchie dès la première ligne. Un code sémantique, performant et sécurisé est le seul garant d'une visibilité durable dans l'écosystème web moderne.

V. Sujet de TD : Maîtriser les Fondamentaux de Laravel

Découverte de l'écosystème Laravel, ce framework n'est pas qu'une simple boîte à outils ; c'est un cadre rigoureux conçu pour industrialiser la production d'applications web modernes, garantissant à la fois sécurité native et scalabilité.

A. 1. Introduction à l'Architecture Web et au Framework Laravel

Le développement web contemporain repose sur une architecture client-serveur complexe. Face à l'explosion du trafic mondial — où des acteurs comme Google captent des volumes de données colossaux — l'utilisation d'un framework comme Laravel permet de gérer cette complexité avec une productivité accrue.

1. Analyse du cycle de vie d'une requête

Pour maîtriser Laravel, vous devez comprendre le cheminement d'une information. Lorsqu'un utilisateur saisit une URL, plusieurs étapes invisibles se succèdent : la résolution DNS trouve l'adresse du serveur, une connexion sécurisée TCP/TLS est établie, puis une requête HTTP (ex: `GET /produits`) est envoyée. Laravel intercepte cette requête via son point d'entrée unique (`public/index.php`). C'est ici que la magie opère : le framework analyse l'URL, interroge la base de données via SQL si nécessaire, et orchestre la génération d'une réponse HTML ou JSON que le navigateur rendra visuellement via le DOM et le CSS.

2. Concept de Base : Le pattern MVC (Model-View-Controller)

Inspiré par des décennies de génie logiciel, le pattern **MVC** structure votre code en trois piliers :

- **Modèle (Model) :** Gère les données et la logique métier (la "vérité" de l'application).
- **Vue (View) :** S'occupe de l'affichage et de l'interface utilisateur.
- **Contrôleur (Controller) :** Le traducteur. Il reçoit les ordres de la Vue, demande les données au Modèle, et renvoie le résultat à la Vue.

Transition : Une architecture élégante nécessite une fondation technique saine ; tout commence par votre environnement de travail.

B. 2. Étape 1 : Initialisation et Configuration de l'Environnement

La cohérence de l'environnement de développement est vitale pour éviter le syndrome du "ça marche sur ma machine". Un environnement standardisé assure que le passage du développement à la production se fait sans friction.

1. Installation pratique

Nous utilisons **Composer**, le gestionnaire de dépendances incontournable de PHP. Ouvrez votre terminal et lancez la création de votre premier projet :

```
# Installation du projet Laravel
composer create-project laravel/laravel mon-application-pedagogique
```

2. Gestion de la configuration et sécurité

Une fois le projet créé, localisez le fichier `.env` à la racine. Ce fichier stocke vos variables d'environnement (identifiants de base de données, clés secrètes).

- **Règle d'or :** Le fichier `.env` ne doit **jamais** être envoyé sur Git (il est listé par défaut dans `.gitignore`). Sa fuite compromettrait l'intégralité de votre sécurité.

3. Concept de Base : Les Dépendances

Une application moderne réutilise des bibliothèques éprouvées. Composer orchestre ces "dépendances" pour garantir qu'elles sont compatibles entre elles, vous permettant de vous concentrer sur la valeur ajoutée de votre code.

Transition : Votre environnement est prêt. Donnons maintenant une structure durable à vos données.

C. 3. Étape 2 : Conception de la Base de Données et Modélisation

La scalabilité d'une application dépend de la rigueur de son schéma de données. Laravel utilise les **Migrations**, un système de versioning pour votre base de données, permettant de modifier la structure sans écrire une seule ligne de SQL manuel.

1. Mise en pratique : Création d'un module de tâches

Imaginons une application de gestion de produits. Exécutez ces commandes :

```
# Générer la migration et le modèle simultanément
php artisan make:model Produit -m
```

Dans le fichier de migration généré (dans `database/migrations`), définissez vos colonnes :

```
Schema::create('produits', function (Blueprint $table) {
    $table->id();
    $table->string('nom');
    $table->decimal('prix', 8, 2);
```

```
    $table->timestamps();
});
```

2. L'ORM Eloquent et la sécurité

Eloquent transforme vos tables en objets PHP. Au lieu d'écrire `SELECT * FROM produits`, vous écrirez `$produits = Produit::all();`.

- **Protection critique :** En utilisant Eloquent, Laravel protège automatiquement votre application contre les **injections SQL** en préparant systématiquement les requêtes.

Transition : Avec des données structurées, nous devons définir la logique permettant à l'utilisateur d'interagir avec elles.

D. 4. Étape 3 : Logique Applicative (Routage et Contrôleurs)

Le routage définit la clarté de vos URLs, un facteur essentiel pour l'expérience utilisateur (UX) et le **SEO**. Une URL comme `/produits/iphone-15` est bien mieux indexée par les moteurs de recherche qu'un paramètre complexe.

1. Définition des routes et logique

Modifiez le fichier `routes/web.php` pour lier une URL à une action :

```
use App\Http\Controllers\ProduitController;
use Illuminate\Support\Facades\Route;

// Une route sémantique et claire
Route::get('/produits', [ProduitController::class, 'index']);
```

Générez ensuite votre contrôleur :

```
php artisan make:controller ProduitController
```

Dans le contrôleur, respectez le principe **DRY (Don't Repeat Yourself)**. Chaque méthode doit avoir une responsabilité unique.

2. Concept de Base : Verbes HTTP et Sémantique

- **GET :** Pour récupérer des données (lecture seule).
- **POST :** Pour soumettre des données (création). C'est impératif pour la sécurité, car les données ne transitent pas de manière visible dans l'URL.

Transition : La logique est prête ; habillons-la maintenant d'une interface accessible et interactive.

E. 5. Étape 4 : Interface Front-End et Interactions

L'interface est le point de contact final. Selon les critères d'ergonomie de **Bastien et Scapin**, une interface doit privilégier le **guidage** (inciter l'utilisateur à l'action par des indices visuels) et réduire la **charge de travail** cognitive.

1. Structure sémantique avec Blade

Utilisez le moteur de template **Blade** pour créer une mise en page cohérente dans `resources/views/layouts/app.blade.php` :

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Ma Boutique</title>
</head>
<body>
    <header role="banner"> <!-- Balise sémantique HTML5 -->
        <nav> <!-- Navigation principale -->
            <ul><li>Accueil</li></ul>
        </nav>
    </header>

    <main>
        @yield('content')
    </main>

    <footer>&copy; 2025 Laravel Expert</footer>
</body>
</html>
```

2. Ergonomie et Sécurité des formulaires

Pour garantir une lecture fluide, respectez la limite de **90 caractères maximum par ligne** de texte et ne dépassez pas **7 éléments** dans vos menus de navigation.

Lorsqu'un utilisateur remplit un formulaire, vous **devez** inclure la protection contre les attaques CSRF :

```
<form method="POST" action="/produits">
    @csrf <!-- Instruction de sécurité critique -->
    <label for="nom">Nom du produit :</label>
```

```
    <input type="text" id="nom" name="nom" required
placeholder="Ex: iPhone 15">
    <button type="submit">Ajouter</button>
</form>
```

3. Concept de Base : Responsive Web Design

Grâce aux **Media Queries**, votre site s'adapte à tous les écrans. C'est un pilier de l'accessibilité mobile (a11y), garantissant que l'interface reste utilisable même sur un smartphone de 480px de large.

Transition : Votre application est fonctionnelle. Passons à la validation finale de sa robustesse.

F. 6. Conclusion et Checklist de Validation

Félicitations, vous avez bâti les fondations d'une application professionnelle. Le développement web est un apprentissage continu, mais la rigueur architecturale reste votre meilleure alliée.

1. Checklist de Sécurisation et de Qualité

Avant toute mise en ligne, validez systématiquement ces points :

1. **Validation rigoureuse :** Ne faites jamais confiance aux entrées utilisateur.

```
$request->validate([
    'nom' => 'required|max:255|string',
    'prix' => 'required|numeric|min:0',
]);
```

1. **Protection XSS :** Utilisez toujours les doubles accolades `{{ $variable }}` de Blade, qui échappent automatiquement le contenu.
2. **Règle des 3 clics :** Un utilisateur doit pouvoir accéder à n'importe quelle information clé en maximum 3 clics depuis l'accueil.
3. **Accessibilité (a11y) :** Vérifiez la présence des attributs `alt` sur vos images et le contraste des couleurs.

Une application est considérée comme un **Produit Minimum Fonctionnel (MVP)** si son CRUD est opérationnel, sa navigation fluide et sa sécurité de base (CSRF, XSS, Injections SQL) totalement verrouillée. Continuez à explorer des concepts comme les API REST et la CI/CD pour parfaire votre expertise. *Bon code !*

VI. TD API platform : Construction d'une Application Full-Stack avec API Platform

A. 1. Introduction et Fondamentaux de l'Architecture Web

Le paysage technologique moderne a opéré une transition majeure des architectures monolithes vers des modèles découplés. Cette évolution privilégie l'usage de **Single Page Applications (SPA)** ou d'applications mobiles communiquant avec des **API** (Application Programming Interfaces). API Platform s'est imposé comme le standard industriel pour le développement rapide, permettant de générer une API robuste, documentée et sécurisée en un temps record tout en respectant les standards du Web.

1. Analyse Stratégique : Monolithe vs Architecture Découplée

Le choix architectural influence directement la **scalabilité** et la maintenance à long terme. Contrairement au monolithe où le code est entrelacé, le découpage permet de faire évoluer le Front-End et le Back-End de manière indépendante.

- **REST :** Préféré pour des contrats d'interface simples, cohérents et une mise en cache native efficace.
- **GraphQL :** Idéal pour des besoins Front-End complexes. Comme illustré dans le [SOURCE_IMAGE_20], GraphQL permet au client de demander précisément les champs nécessaires (ex: `name`, `order`) via un point d'entrée unique, évitant l'over-fetching de données inutiles.

2. Concepts de Base

- **Modèle Client-Serveur :** Séparation des responsabilités entre l'interface utilisateur et le traitement des données.
- **Protocole HTTP/HTTPS :** Standard de transfert sécurisé indispensable en production.
- **Cycle de vie d'une requête (Réf. [SOURCE_IMAGE_2]) :**
    1. **URL saisie → DNS :** Première étape cruciale de résolution de nom.
    2. **Connexion TCP/TLS :** Établissement du tunnel sécurisé.
    3. **Requête HTTP :** Envoi de la méthode (GET/POST) et des headers.
    4. **Traitement Serveur & Requête SQL :** Interaction avec la persistance.
    5. **Réponse DB :** Retour des données brutes.
    6. **Réponse HTTP (HTML/JSON) :** Livraison de la ressource.
    7. **Rendu DOM :** Construction visuelle par le navigateur.

La maîtrise de ces bases est le prérequis indispensable avant de passer à la phase technique d'installation.

B. 2. Initialisation de l'Environnement et Installation

Pour garantir l'homogénéité des environnements (le fameux "ça marche sur ma machine"), API Platform s'appuie sur la conteneurisation via **Docker**. Cette approche permet d'isoler chaque service de la pile technique dans un environnement contrôlé et reproductible.

1. Guide Technique d'Installation

1. **Docker :** Pivot central de l'architecture, il encapsule les services sans polluer le système hôte.

2. **HTTPS dès le développement :** Utilisation de certificats locaux pour simuler les conditions réelles de production et sécuriser les échanges.
3. **Secrets & Environnement :** Isolation des accès (mots de passe DB, clés JWT) via des variables d'environnement pour respecter les standards de sécurité industrielle.

2. Concepts de Base

- **Docker :** Garantit la cohérence entre le développement et la production.
- **PHP (Back-End) :** Moteur d'exécution de la logique applicative Symfony.
- **Serveur Web (Caddy/Nginx) :** Gère la réception des requêtes et la distribution vers PHP.

Une fois l'environnement prêt, l'étape suivante consiste à structurer les données que l'application va manipuler.

C. 3. Modélisation des Données et Persistance (ORM)

Une modélisation rigoureuse est le fondement d'une **Clean Architecture**. L'**ORM (Object-Relational Mapping)** agit comme un traducteur entre vos objets métier et les tables de votre base de données. En tant qu'architectes, nous appliquons ici les principes **SOLID**, notamment le principe de responsabilité unique, pour concevoir des entités pérennes.

1. Mise en Pratique : Création d'Entités

En API Platform, une "Entité" (ex: un Produit) devient une ressource API par simple ajout d'attributs PHP 8. Cette approche suit le principe **DRY (Don't Repeat Yourself)** : la définition de vos données génère automatiquement les points d'entrée de votre API.

2. Analyse Stratégique : Performance et Persistance

Le choix du moteur de base de données dépend de la nature des relations. Une structure bien indexée est vitale pour la rapidité des requêtes SQL et la mise en cache.

- **Relationnelle (SQL) :** PostgreSQL ou MySQL pour des données structurées nécessitant une forte cohérence.
- **NoSQL :** MongoDB pour des documents flexibles et des volumes massifs de données non structurées.

3. Concepts de Base

- **PostgreSQL / MySQL :** Systèmes de gestion de bases de données relationnelles standards.
- **MongoDB :** Exemple phare de base de données NoSQL orientée documents.
- **Indexation :** Technique d'optimisation permettant au serveur de trouver une donnée sans scanner toute la table.

La structure étant définie, nous devons maintenant implémenter la logique permettant d'interagir avec ces données.

D. 4. Logique Applicative : Opérations et Contrôleurs

L'interaction avec les ressources suit le cycle **CRUD** (Create, Read, Update, Delete). API Platform maximise l'efficacité en automatisant ces opérations, nous permettant de nous concentrer sur la logique métier complexe plutôt que sur le code "plomberie".

1. Développement et Routage Déclaratif

Le routage moderne s'appuie sur une approche déclarative. Tout comme le décorateur `@app.route` de Flask en Python permet de lier une fonction à une URL, API Platform utilise les **Attributes PHP** pour définir ses endpoints. Cette méthode permet de découpler proprement la configuration de l'implémentation, facilitant la lecture et la maintenance (principe **KISS : Keep It Simple, Stupid**).

2. Validation : Le miroir de sécurité

Il est impératif que la validation **Client-Side** (HTML5) soit systématiquement doublée par une validation **Server-Side** (API Constraints) pour garantir l'intégrité des données face aux injections ou aux requêtes malveillantes.

Contrainte	Standard Source	Exemple d'application
Required	`required` HTML5	Email, Nom d'utilisateur obligatoire
Pattern	`pattern` (Regex)	Mot de passe complexe : `(?=.*\d)(?=.*[a-z])`
Min/Max	`min`, `max`	Quantité en stock (ex: entre 1 et 5)
Type	`type="email/url"`	Format d'adresse mail ou URL valide

3. Concepts de Base

- **200 OK / 201 Created :** Succès de l'opération.
- **400 Bad Request :** Erreur de validation (données invalides).
- **401 Unauthorized :** Authentification manquante.
- **404 Not Found :** Ressource inexistante.
- **500 Internal Server Error :** Crash ou erreur logique du serveur.

L'API étant fonctionnelle, elle doit maintenant servir de moteur à une interface utilisateur interactive.

E. 5. Interaction avec le Front-End et Expérience Utilisateur (UX)

Le Front-End consomme l'API via **JavaScript (ES6+)**. L'usage des **Promesses** et de la syntaxe `async/await` est indispensable pour gérer les appels réseau de manière non-bloquante, assurant une interface fluide.

1. Accessibilité et Structure Sémantique

Une interface professionnelle doit être inclusive. Nous utilisons les balises sémantiques HTML5 issues du modèle de structure (Réf. [SOURCE_IMAGE_12]) : `<header>`, `<nav>`, `<main>`, `<article>`, `<aside>`, et `<footer>`. L'ajout d'attributs **WAI-ARIA** (ex: `aria-label`) garantit que l'application est exploitable par les technologies d'assistance (lecteurs d'écran).

2. Analyse Stratégique : Les Critères de Bastien & Scapin

L'ergonomie ne se limite pas à l'esthétique ; elle repose sur des critères scientifiques :

1. **Guidage :** Utilisation de *placeholders* et d'indices visuels pour orienter l'utilisateur.
2. **Charge de travail :** Application de la **règle des 3 clics** (accès rapide à l'info) et brièveté des textes.
3. **Feedback immédiat :** Message de confirmation (ex: "Profil mis à jour") après chaque action.

3. Pro-Tips Pédagogiques

- **Règle des 90 caractères :** Ne dépassez jamais 90 caractères par ligne pour préserver le confort de lecture.
- **Hiérarchie Visuelle :** Utilisez les balises `<h1>` à `<h6>` pour structurer l'importance de l'information.

L'application doit désormais recevoir une couche de protection robuste contre les menaces courantes.

F. 6. Sécurisation, SEO et Finalisation

La mise en production exige une vigilance accrue. La sécurisation repose sur deux piliers : le **CORS** (Cross-Origin Resource Sharing) pour autoriser uniquement votre Front-End à appeler l'API, et l'authentification par **JWT** (JSON Web Token) pour des échanges d'identité sécurisés et stateless.

1. Checklist de Mise en Production (Standards Industriels)

- [ ] **Sécurité :** Audit face au **OWASP Top 10** (injections, authentifications rompues, etc.).
- [ ] **Protection :** Mise en œuvre de protections contre les failles **XSS** et **CSRF**.
- [ ] **SEO :** Optimisation des balises *Title* (moins de 100 char) et *Meta Description*.
- [ ] **Accessibilité :** Validation via les standards **WAI-ARIA** et outils comme Lighthouse.
- [ ] **Performance :** Minification des assets et mise en place du *Lazy loading*.

2. Analyse "So What?" : Performance et Business

L'optimisation des **Core Web Vitals** est le nerf de la guerre commerciale :

- **LCP (Largest Contentful Paint) :** Vitesse d'affichage du contenu principal.
- **FID (First Input Delay) :** Réactivité de l'interface au premier clic.
- **CLS (Cumulative Layout Shift) :** Stabilité visuelle lors du chargement. Un mauvais score ici dégrade votre référencement Google et fait fuir 75 % des utilisateurs qui ne cliquent jamais au-delà de la première page.

3. Concepts de Base

- **Robots.txt :** Fichier à la racine indiquant aux crawlers quelles zones ignorer (ex: `/admin`).
- **Sitemap.xml :** Cartographie complète de vos URLs pour faciliter l'indexation par les moteurs.

Ce projet valide l'acquisition d'un spectre complet de compétences, de l'architecture logicielle (SOLID, DRY) à l'optimisation de l'expérience utilisateur, faisant de vous des développeurs Full-Stack prêts pour le marché industriel.

VII. Sujet du TP noté :

Lors de la dernière séance de nos cours, vous aurez 6minutes pour présenter sous forme de démonstration et de revue de code un jeu simple codé en javascript. Ce jeu, je le répète simple, doit être codé à partir d'un code 100% perso, sans IA et à 100% en JS, l'utilisation de code existant (dépôt de code, librairie, etc.) est proscrit.
Niveau fonctionnalité : faire la gestion des comptes (Log In) et des scores en en PHP sans framework.
La partie CSS du front peut utiliser un framework.
Essayez de mettre en ligne votre jeu et de le partager et le rendre fonctionnel sous le discord de la promo.